Statistical Methods in AI - Monsoon 2026

Assignment 3

Deadline: 9 October 2026, 11:59 P.M.

Instructors: Prof Saikiran Bulusu, Prof Anoop M. Namboodiri

General Instructions

- Your assignment must be implemented in Python.
- Clearly label and organize your code, including comments that explain the purpose of each section and key steps in your implementation. Ensure that your files are well-structured, with headings, subheadings, and explanations as necessary.
- Make sure to test your code thoroughly before submission to avoid any runtime errors or unexpected behavior.
- Your assignment will be evaluated not only based on correctness but also on the quality of code, the clarity of explanations, and the extent to which you've understood and applied the concepts covered in the course.
- We are aware of the possibility of submitting the assignment late in GitHub Classroom using various hacks. Note that we will have measures in place accordingly and anyone caught attempting to do the same will be given a straight zero in the assignment.

AI Tools Usage Instructions (Mandatory if Applicable)

We are aware how easy it is to write code and solve questions with the help of LLM services, but we strongly encourage you to figure out the answers yourself. If you use any AI tools such as ChatGPT, Gemini, Claude, etc., to assist in solving any part of this assignment:

- **If you are unable to explain any part of the solution/code during evaluations, that solution/code will be considered plagiarized and you will be penalized.**You must also be able to briefly explain how you modified or verified the AI-generated content.
- You must include a **shareable link** to the AI Tool chat history or a **screenshot of the relevant conversation**. Do **NOT** share any private or sensitive personal information in the AI Tool conversations you include.

Submission Instructions

- Submit a self-contained Jupyter notebook containing all code, plots, and printed tables.
- Report all the analysis, comparison and any metrics in the notebook or a separate report that is part of the submission itself. No external links to cloud storage files, wandb logs or any other alternate will be accepted as part of your submission. Only the values and visualizations as part of your commits will be graded.
- Use your institute email ID to generate a personalized random seed:
  - Get the part before @ in your IIITH email (e.g., username from username@program.iiit.ac.in).
  - Use the SHA-256 hash of this string to ensure uniqueness:

```
import hashlib
seed = int(hashlib.sha256(username.encode()).hexdigest(), 16) % (2**32)
```

  - Use this seed in all random number generators (e.g., `np.random.default_rng(seed)`).

- All plots must include your email username in the title or filename.

```
plt.text(
    0.95, 0.95, "username",
    ha='right', va='top',
    transform=plt.gca().transAxes,
    fontsize=10, color='gray', alpha=0.7
)
```

1 Census Income Prediction with Random Forests [100 marks]

Random Forests are ensemble learning methods that combine multiple decision trees using bootstrap sampling and random feature selection. Although Random Forests are available as standard machine learning libraries, the purpose of this assignment is to understand and implement the algorithm from scratch. In this assignment, you will build a complete Random Forest classifier without using pre-existing implementations of decision trees or Random Forests.

Dataset: UCI Adult Census Income dataset [Becker, B. and Kohavi, R. (1996). Adult [Dataset]. UCI Machine Learning Repository. DOI: 10.24432/C5XW20.]

The dataset contains approximately 48,842 instances and 14 input features. The task is to predict whether an individual's annual income is $\leq$ 50K or $>$ 50K. The dataset contains both categorical and numerical attributes, as well as missing values. The 14 attributes in the dataset are: age, workclass, fnlwgt (final weight), education, education-num, marital-status, occupation, relationship, race, sex, capital-gain, capital-loss, hours-per-week, native-country. The target variable is income, with two classes: $\leq$ 50K or $>$ 50K.

1.1 Data Preprocessing [9 marks]

1.1.1 Dataset Exploration [3 marks]

Explore the dataset and report:

- number of rows and columns,
- data type of every feature,
- number of unique values for categorical features,
- number of missing values in each feature,
- number and proportion of examples in each target class.

Choose and justify a missing value strategy. Any statistics used for imputation must be computed from the training set only.

1.1.2 Redundant Features [2 marks]

For numerical features, compute and visualise the Pearson correlation matrix. Identify strongly correlated features and discuss if any should be removed.
Note: You may remove features identified as redundant, in which case clearly state which features you are removing. The resulting number of features p should be used consistently throughout the Random Forest experiments.

1.1.3 Train/Validation/Test Split [2 marks]

Use the official `adult.test` file as the final test set. Split `adult.data` into: 80% training data and 20% validation data. The split must be stratified according to the target class. Report the number of samples in each split and the class distribution in each split.

Note: The test set must not be used for model selection or hyperparameter tuning.

1.1.4 Categorical Encoding [2 marks]

Convert categorical variables into a numerical representation suitable for your decision-tree implementation. Use one-hot encoding to:

- Determine the categories using the training set,
- Apply the same representation to the validation and test sets,
- Do not allow information from the validation or test sets to determine the training representation.

Report the final number of features after preprocessing.

1.2 Decision Tree from Scratch [22 marks]

1.2.1 Finding the Best Split [5 marks]

For numerical features, consider candidate thresholds based on sorted unique feature values. For every candidate split:

1. Divide the observations into two children,
2. Calculate the split quality,
3. Reject splits producing an empty child,
4. Retain the best valid split.

1.2.2 Implementation [6 marks]

Build a binary decision tree classifier from scratch. Refer to the starter code for the template of `DecisionTree` class.

A tree node can be represented using a dictionary or a small class. For an internal node, store information like selected feature, threshold, left child and right child. Refer to the starter code for the template of `TreeNode` class.

For a leaf, store the predicted class. A node should become a leaf when an appropriate stopping condition is reached, such as:

- All examples belong to the same class,
- Maximum depth has been reached,
- There are too few samples to split,
- No valid split exists.

The prediction of a leaf should be the majority class of the training examples reaching that leaf.

1.2.3 Tree Depth Experiment [5 marks]

Train decision trees with:

$$\texttt{max_depth} \in \{1, 3, 5, 7, \text{unrestricted}\}.$$

For each setting, report: training accuracy, validation accuracy, number of leaves, actual maximum tree depth and training time.

Plot training and validation accuracy against tree depth.

1.2.4 Gini versus Entropy [6 marks]

Train otherwise identical trees using:

1. Gini impurity,
2. entropy.

Compare training accuracy, validation accuracy, number of leaves, training time.

1.3 Bootstrap Sampling and Bagging [8 marks]

1.3.1 Bootstrap Sampling [5 marks]

Implement bootstrap sampling from scratch. For a dataset containing N observations, create a bootstrap sample by drawing N observations **with replacement**.

Generate 10 bootstrap samples. For each sample, report:

- number of unique observations selected,
- number of observations not selected.

Compare your empirical results with the theoretical approximation:

$$1 - e^{-1} \approx 0.632.$$

Explain why approximately 63.2% of the observations are expected to appear in a bootstrap sample at least once.

1.3.2 Bagged Trees [3 marks]

Construct a bagged-tree ensemble using:

- bootstrap sampling,
- all available features at every split.

Train ensembles with:

$$B \in \{1, 10, 25, 50, 100\}.$$

For each value of B, report validation accuracy and training time. Plot validation error against the number of trees.

1.4 Random Forest from Scratch [32 marks]

1.4.1 Random Feature Selection [10 marks]

Random Forests introduce an additional source of randomness by selecting only a subset of features at each node. At every internal node, randomly select m candidate features and search for the best split only among those features.

Note: The feature subset must be sampled independently at each node. Do not select one feature subset once per tree.

Compare:

$$m \in \left\{1, \sqrt{p}, \frac{p}{2}, p\right\},$$

where p is the final number of features after preprocessing.

Use the same number of trees, maximum depth, minimum leaf size, minimum split size and seed for a fair comparison. For each value of m, report training accuracy, validation accuracy, OOB accuracy, training time.

1.4.2 Implementation [7 marks]

Implement a complete Random Forest classifier from scratch. Each tree must:

1. be trained on a bootstrap sample,
2. keep track of observations that were not selected,
3. randomly select features at every split,
4. be stored as part of the forest.

The forest prediction should be obtained using majority voting.

Refer to the starter code for the template of `RandomForest` class.

1.4.3 Number of Trees Experiment [8 marks]

Train Random Forests with:

$$B \in \{1, 10, 50, 100, 250\}.$$

For each value of B, report training accuracy, validation accuracy and training time.

Produce the following plots:

- validation error vs number of trees,
- training time vs number of trees.

Discuss:

1. How does increasing the number of trees affect performance?
2. How does computational cost scale as the number of trees increases?
3. What value of B would provide a reasonable trade-off between performance and training cost?

1.4.4 Single Tree versus Random Forest [7 marks]

Compare your best single decision tree with your best Random Forest.

Report training accuracy, validation accuracy, training time, model complexity where appropriate. Explain why an ensemble of randomized trees can outperform an individual tree.

1.5 Out-of-Bag Evaluation [8 marks]

For every training observation, identify the trees for which that observation was not included in the bootstrap sample. Use predictions only from these trees to construct an OOB prediction. The OOB prediction for an observation should be the majority vote among the trees for which the observation was OOB. If an observation has no OOB prediction, clearly specify how your implementation handles that case.
Report the distribution of OOB trees available per observation.

Compare the OOB predictions and the validation predictions by reporting:

- OOB accuracy vs. validation accuracy,
- OOB error vs. validation error.

1.6 Bias–Variance and Hyperparameter Study [10 marks]

Investigate the effect of Random Forest hyperparameters.

1.6.1 Effect of Tree Depth [4 marks]

Evaluate:

$$\texttt{max_depth} \in \{2, 4, 6, 10, \text{unrestricted}\}.$$

For every setting, report training error and validation error. Plot both errors against tree depth.

Explain the observed behaviour using the concepts of bias, variance, underfitting and overfitting.

1.6.2 Effect of Minimum Leaf Size [3 marks]

Evaluate:

$$\texttt{min_samples_leaf} \in \{1, 2, 5, 10, 25\}.$$

Report training and validation error. Discuss how increasing the minimum leaf size affects model complexity and variance.

1.6.3 Hyperparameter Selection [3 marks]

Perform a small, manually specified hyperparameter study involving a combination of:

- number of trees,
- maximum depth,
- maximum number of features,
- minimum leaf size.

Select the final model using the validation set. Clearly state which configuration you selected and why.

Note: Do not use the test set for hyperparameter selection.

1.7 Feature Importance and Model Interpretation [6 marks]

Implement two feature-importance methods from scratch.

1.7.1 Impurity-Based Feature Importance [3 marks]

For every split in every tree, record the reduction in impurity produced by the split. Aggregate the reductions across the forest and normalize them so that:

$$\sum_{j=1}^{p} I_j = 1.$$

Report the top 10 features. Produce a bar plot of their importance.

1.7.2 Permutation Importance [3 marks]

Implement permutation importance on the validation set. First compute the baseline validation accuracy. Then, for each feature:

1. randomly permute that feature,
2. predict using the trained Random Forest,
3. measure the decrease in accuracy,
4. restore the feature,
5. repeat the procedure several times.

Rank features according to their average decrease in validation accuracy. Report and plot the top 10 features.

1.8 Final Evaluation and Error Analysis [5 marks]

Only after completing model selection and hyperparameter tuning should you evaluate the final model on the official test set.

Treat `>50K` as the positive class and implement the following metrics from scratch:

- accuracy,
- precision,
- recall,
- F1-score,
- confusion matrix.

Report all metrics on the final test set. Also report the performance of a simple majority-class baseline.

2 SecureVision: Face Compression and Recognition with PCA [100 marks]

You are a machine learning engineer at **SecureVision**, a startup building on-device face verification for smart door locks. The lock's onboard chip has very limited flash storage and compute power, so it cannot store or process raw 64×64 pixel images (4,096 numbers per face) for every enrolled resident.

Your task is to use Principal Component Analysis (PCA) to discover the small set of **Eigenfaces**, the dominant patterns of variation across human faces, and use them to:

1. compress each enrolled face into a small set of numbers instead of all 4,096 pixel values,
2. reconstruct the face from this compressed version and check whether it is still recognizable,
3. measure the trade-off between storage savings and reconstruction quality,
4. decide, from your own experimental results, a suitable number of components k for SecureVision,
5. find out where the eigenface representation holds up and where it fails.

Dataset: Olivetti Faces (Kaggle) — 400 grayscale face images (64×64 pixels each) of 40 distinct people, 10 images per person, pixel values scaled to $[0, 1]$. The same data is also available through `sklearn.datasets.fetch_olivetti_faces()`.

Constraints:

- PCA must be implemented from scratch using NumPy. `sklearn.decomposition.PCA` must **not** be used anywhere in this question.
- `sklearn` may be used for the recognition classifier.
- PCA must be fitted using the training set only. The test set must not be used to compute the mean face, the covariance or Gram matrix, the principal components, the explained variance or any other PCA parameter.
- Wherever you choose a parameter yourself, state the value and briefly justify it.

2.1 Data and Transformed Test Sets [12 marks]

1. Load the dataset and report the number of images, the image size and the number of identities. Display 5 random faces labelled with their identity.
2. Flatten each image to a 4,096-dimensional vector and split the data 80/20 into training and test sets, stratified by identity. Report the random seed and the shape of each split.
3. Build the transformed test sets below. Transform the 64×64 images before flattening, keep values in $[0, 1]$, and fix all random seeds. For each transformation, show the same 3 test faces before and after.
    - **Rotation** about the image centre by $\theta \in \{0°, 15°, 30°, 45°\}$ (one test set per angle). State how you fill the uncovered pixels.
    - **Gaussian noise** with mean 0 and $\sigma \in \{0.05, 0.1, 0.2, 0.5\}$ added to every pixel independently (one test set per σ), then clipped to $[0, 1]$.

- **Illumination:** multiply each image by a horizontal gradient rising linearly from $1-s$ (left edge) to $1+s$ (right edge), with $s \sim \mathcal{U}(0.3, 0.9)$ drawn per image, then clip to $[0, 1]$. Make an illuminated copy of both the test **and** training sets.

Generate each transformed set once here, using your personalised seed (so each image gets one fixed value of s and each noisy set one fixed noise draw), and reuse these same sets in every later question. Do not regenerate them.

All experiments on transformed sets use $k_T = 50$ components and the PCA and classifier fitted on the **clean** training set, unless stated otherwise.

2.2 PCA from Scratch [22 marks]

Implement the class below, where X_c is the training data minus the training mean face. `"direct"` eigendecomposes the $D \times D$ covariance $C = X_c^\top X_c/(N-1)$; `"gram_trick"` eigendecomposes the $N \times N$ matrix $L = X_c X_c^\top/(N-1)$ and recovers unit-length eigenvectors of C from it. Store the mean face, the components sorted by decreasing eigenvalue, the eigenvalues and the explained variance ratios. `k=None` keeps all components with non-zero eigenvalue.

```
class PCAFromScratch:
    def __init__(self, method="direct"): ... # "direct" or "gram_trick"
    def fit(self, X, k=None): ... # X: (N, D); keep the top k components
    def transform(self, X, k=None): ... # X: (M, D) -> Z: (M, k)
    def inverse_transform(self, Z): ... # Z: (M, k) -> X_hat: (M, D)
```

1. Time `fit` for both methods on the training set and explain the difference using the matrix sizes.
2. For the top 50 components, report the largest absolute eigenvalue difference between the two methods and the absolute correlation between each pair of corresponding components (signs are arbitrary). State which method you use from here on.
3. Display the mean face and the top 15 components as 64×64 images.

2.3 Compression and Reconstruction [24 marks]

1. Derive the number of values needed to store N faces of dimension D with k components so they can be reconstructed, with the mean face and components stored once. Implement the function below; use $N = 320$ and $D = 4096$ throughout.

```
def compression_ratio(N, D, k):
    """Raw storage (N * D) divided by compressed storage."""
    ...
```

2. For 3 clean test faces, show the original and its reconstructions at $k \in \{10, 50, 100\}$. Which details disappear first as k decreases, and why?
3. Implement reconstruction MSE: the mean squared pixel difference over all images and pixels. Then:
   - plot clean test MSE and compression ratio against $k = 1, \ldots, 200$, and tabulate both for $k \in \{10, 50, 100\}$;
   - from the training set, plot the explained variance ratio of the first 200 components (scree plot) and the cumulative explained variance, and report the smallest k reaching 90%;

- state what the MSE curve and scree plot imply for choosing k.

4. At k_T:
   - **Rotation:** plot MSE (rotated face vs. its reconstruction) against the four specified angles.
   - **Noise:** plot against the four specified σ values (a) MSE of noisy vs. clean faces and (b) MSE of reconstructed noisy vs. clean faces. Explain why they differ.
   - **Illumination:** report MSE (illuminated face vs. its reconstruction) next to the clean test MSE.

2.4 Face Recognition [24 marks]

Choose an `sklearn` classifier and distance metric (e.g. k-NN, Euclidean) and justify it. Keep the classifier and its hyperparameters fixed for all experiments. Always fit it on training faces and test on test faces represented the same way (raw pixels, or the same PCA and k). Accuracy is the fraction of test faces correctly identified.

1. Tabulate clean test accuracy on raw pixels and on PCA features for $k \in \{5, 10, 20, 50, 100\}$.
2. For these k, plot clean test MSE and accuracy on one figure (two y-axes). Does lower MSE always mean higher accuracy?
3. At k_T, plot accuracy against the four specified rotation angles and against the four specified noise σ values.
4. At k_T, tabulate accuracy for: (a) clean PCA and classifier, clean test set; (b) clean PCA and classifier, illuminated test set; (c) PCA and classifier refitted on the illuminated training set, illuminated test set.

2.5 Choosing k for SecureVision [12 marks]

Implement the function below. The inputs are equal-length lists, where index i holds the clean test results for `k_values[i]`. A k qualifies if its accuracy is at least max(accuracy)$-$ `max_accuracy_drop` and its ratio is at least `min_ratio`.

```
def recommend_k(k_values, mse, accuracy, ratio, max_accuracy_drop, min_ratio):
    """Return the smallest qualifying k, or None if no k qualifies."""
    ...
```

Use $k \in \{5, 10, 20, 50, 100\}$ and your results from Face Recognition.

1. With `max_accuracy_drop = 0.02` and `min_ratio = 10`, report the chosen k with its MSE, accuracy and ratio. If none qualifies, state which condition each k fails.
2. Change one threshold, rerun, and explain why the recommended k did or did not change.

2.6 Where PCA Breaks [6 marks]

For rotation, noise and illumination, use your results at k_T to state whether each increased MSE and whether it reduced accuracy relative to the clean test set. Explain why, and propose one fix for each.